

Sistema de Pase de Lista Automático por Reconocimiento de Marcha

Reporte de Avance del Proyecto

Visión por Computadora · Biometría por Marcha (Gait Recognition)

Integrantes
Aguilar Felix, Jesús Antonio
Cazarez Lara, Jesús Kevin
Espinoza Mendivil, Jorge Alberto
Vázquez Cárdenas, César Enrique

Fecha de elaboración: 17 / 05 / 2026

Fase actual: 6.5 (validación) · 7.0 (optimización completada)

1. Resumen ejecutivo

El proyecto busca construir un sistema automático de pase de lista basado en **reconocimiento de marcha (gait recognition)**: identificar a cada persona que pasa frente a una cámara fija mediante el patrón temporal de su caminar, y detectar personas *fuera de la lista* (problema **open-set**).

A la fecha hemos completado el pipeline end-to-end: detección, segmentación, extracción de embedding y matching contra galería, todo corriendo en vivo a **20.7 FPS** sobre GTX 1650 (4 GB). El modelo en producción es **GaitBase** fine-tuneado multisesión sobre nuestro dataset propio de **19 sujetos** a 90° lateral. La etapa abierta es la validación del nuevo umbral $\tau=0.85$ en condiciones en vivo. Como bloque final se propone incorporar **algoritmos genéticos / estrategias evolutivas** para la optimización conjunta de hiperparámetros del pipeline.

Indicador	Valor
Sujetos enrolados	19
FPS efectivos en vivo	20.7 (+21 % vs baseline)
Umbral activo (τ)	0.85 (recalibrado para vivo)
EER open-set (val)	8.11 %
TAR@FAR=0 % (val)	86.49 %
Hardware actual	GTX 1650 4 GB · i5-12450H · 16 GB RAM
Cámara	Logitech C920 (USB)

2. Estado actual del proyecto

2.1 Fases completadas

Fase	Descripción	Estado
0	Setup, descubrimiento y stack validado	Cerrada
1	Auditoría del dataset (19 sujetos, 90° lateral, 2 sesiones)	Cerrada
2	Preparación, normalización y splits subject-disjoint	Cerrada
3	Baseline GaitBase + fine-tuning monosesión	Cerrada
3.5	Fine-tuning multisesión (checkpoint en producción)	Cerrada
4	Multimodal (silueta + pose) — cierre como Caso C	Cerrada
5	Open-set: τ calibrada, EER y TAR@FAR reportados	Cerrada
6	Pipeline de inferencia en vivo (script ejecutable)	Cerrada
6.5	Validación end-to-end con τ recalibrada	En curso
7.0	Optimización de FPS y fix CUBLAS	Cerrada
7.1	SQLite + documentación + GA/CMA-ES	Pendiente

2.2 Logros técnicos recientes (mayo 2026)

- **Fix CUBLAS:** resuelto `CUBLAS_STATUS_EXECUTION_FAILED` en GTX 1650 desactivando autocast FP16 (la GTX 1650 no tiene Tensor Cores). El embedder ahora corre estable en FP32.
- **Optimización de FPS:** imgsz YOLO reducido de 1280×720 a 640. Resultado: **+21 % de throughput** (de 17.1 a 20.7 FPS efectivos).
- **Recalibración de umbral:** τ bajado de 0.9847 a **0.85** tras observar que enrolados producían similitud ≈ 0.95 en vivo pero eran rechazados por el umbral derivado del set de validación.
- **Profiling per-etapa:** con `torch.cuda.Event`; ahora sabemos que RVM (segmentación) es el nuevo cuello de botella (~35 % del tiempo de frame).

2.3 Métricas de rendimiento del pipeline (post-optimización)

Etapas	Tiempo (ms)	% del frame	Comentario
YOLO (detección)	17.98	23 %	imgsz=640 aplicado
RVM (segmentación)	26.45	35 %	Cuello de botella actual
GaitBase (embedding)	59.48	8 % amortizado	Solo en cierre de ventana
Matcher	0.28	<1 %	CPU, negligible
FPS efectivos	20.7	—	+21 % vs baseline

3. Pendientes por terminar

3.1 Validación end-to-end (Fase 6.5 — alta prioridad)

- Correr `infer_live.py` con $\tau=0.85$ sobre los 19 sujetos enrolados y confirmar que aparecen como **PRESENTE**, registrando similitud observada.
- Probar con personas **fuera de la galería** y confirmar que se rechazan como **UNK** (open-set).
- Pruebas de robustez en al menos 4 condiciones: cambio de ropa, velocidad de caminata distinta, iluminación variable, oclusión parcial.
- **Métrica de éxito:** $\tau=0.85$ acepta a todos los enrolados y rechaza al 100 % de los no enrolados, con margen de seguridad ≥ 0.05 .

3.2 Persistencia y trazabilidad (Fase 7.1)

- Migrar el log de asistencia de **CSV** → **SQLite** para soportar consultas, audit trail y exportes filtrados (por sujeto, fecha, rango horario).
- Esquema mínimo: tabla events (timestamp, subject_id, similarity, decision, session_id) + tabla gallery (subject_id, enroll_date, template_path).
- Backups automáticos diarios.

3.3 Optimización de hiperparámetros con GA / ES (Fase 7.2)

- Implementar el bloque descrito en la sección 4 de este reporte.
- Definir presupuesto de evaluaciones (≈ 200 individuos $\times$ 30 generaciones).
- Validar mejora real sobre el conjunto de val antes de promover a producción.

3.4 Documentación final y entrega

- Manual de operación (cómo enrolar, cómo correr, cómo interpretar el log).
- Guía de calibración de τ para una cámara nueva.
- Reporte técnico final con métricas consolidadas.
- Defensa / presentación con demo en vivo.

3.5 Mejoras opcionales (Fase 8 — si hay tiempo)

- Reemplazar RVM por un segmentador más ligero (SegFormer-B0 o YOLOv8-seg) para subir a 30 FPS sostenidos.
- Exportar el embedder a **ONNX** o **TensorRT** para despliegue en hardware modesto (laptop sin GPU dedicada).
- Ampliar el dataset: capturar a 0° , 45° y 135° para mejorar la generalización.

4. Optimización de hiperparámetros con Algoritmos Genéticos / Estrategias Evolutivas

4.1 Justificación

El pipeline expone una decena de hiperparámetros cuya interacción **no es diferenciable** respecto a la métrica final (TAR@FAR=1 % en open-set). Búsqueda en malla (*grid search*) es prohibitiva por la combinatoria, y *random search* ignora dependencias entre parámetros. Por eso proponemos un esquema **basado en población**:

- **Algoritmo Genético (GA)** con codificación mixta (real + entera), cruce *simulated binary crossover* (SBX) y mutación polinomial.
- **CMA-ES** (Covariance Matrix Adaptation Evolution Strategy) para los subespacios puramente continuos: aprende correlaciones entre parámetros y es el referente moderno en optimización *black-box*.
- Comparativa entre ambos: GA explora mejor espacios mixtos; CMA-ES converge más rápido en lo continuo. Reportaremos ambos y elegiremos el ganador en val.

4.2 Hiperparámetros candidatos

Parámetro	Tipo	Rango propuesto	Impacto esperado
τ (umbral open-set)	real	[0.70, 0.95]	TAR / FAR
ventana (frames)	entero	[30, 90]	Estabilidad embedding
stride (frames)	entero	[10, 45]	Latencia / cobertura
N (confirmaciones)	entero	[1, 4]	Robustez vs latencia
imgsz YOLO	discreto	{416, 512, 640, 736}	FPS / detección
IoU NMS YOLO	real	[0.30, 0.70]	Tracking estable
conf YOLO	real	[0.25, 0.60]	Precisión / recall
bbox padding silueta	real	[1.00, 1.30]	Calidad de silueta
agregación embedding	discreto	{mean, max, attn}	Robustez temporal

4.3 Función de fitness

El fitness combina **desempeño biométrico** y **velocidad**, evitando soluciones que sólo optimizan una dimensión:

$$fitness = w_1 \cdot \text{TAR@FAR=1 \%} - w_2 \cdot (1 - \text{rank-1}) - w_3 \cdot \max(0, 15 - \text{FPS}) / 15$$

- **$w_1=0.6$** : peso del open-set (objetivo principal).
- **$w_2=0.3$** : peso del closed-set (rank-1).
- **$w_3=0.1$** : penalización si FPS < 15 (requisito de tiempo real).
- Evaluación sobre **val subject-disjoint**; el test queda intocable para el reporte final (evita sobreajuste por búsqueda).

4.4 Configuración propuesta de la búsqueda

Parámetro de la búsqueda	Valor
--------------------------	-------

Tamaño de población	30 individuos
Número de generaciones	30
Presupuesto total de evaluaciones	≈ 900 (cacheable hasta 60–70 %)
Operador de cruce (GA)	SBX, $\eta_{\text{c}}=15$
Operador de mutación (GA)	Polinomial, $\eta_{\text{m}}=20$, $p_{\text{m}}=1/D$
Estrategia (ES)	CMA-ES con $\sigma_0=0.3$
Selección	Torneo binario (GA) / (μ, λ) (ES)
Elitismo	Sí, 2 mejores (GA)
Semilla	Fija para reproducibilidad (seed=42)
Librería sugerida	DEAP (GA) + pycma (CMA-ES)

4.5 Plan de implementación (estimado: 1–2 semanas)

- **Día 1–2:** envolver el pipeline en una función `evaluate(individual) → fitness` determinista.
- **Día 3:** integrar DEAP y correr GA piloto (10 generaciones, 10 individuos).
- **Día 4:** integrar pycma y correr CMA-ES piloto en el subespacio continuo.
- **Día 5–7:** correr la búsqueda completa (usar la segunda PC RTX 2060).
- **Día 8–10:** validar el mejor individuo en test y documentar.

Nota técnica: dado que el embedding es la operación más cara, cachearemos embeddings por (*checkpoint*, *frame_set*) para que las evaluaciones que sólo cambian τ , ventana o stride reutilicen vectores ya calculados. Esto reduce el costo efectivo de cada evaluación en ~ 70 %.

5. Material y hardware a comprar

Este apartado lista lo que recomendamos adquirir para cerrar el proyecto con calidad de entrega y soportar las fases pendientes (validación robusta, búsqueda evolutiva, demo final). Los precios son **orientativos en MXN** (mayo 2026) y deben confirmarse en cotización.

5.1 Indispensable (sin esto, no se cierra el proyecto)

Ítem	Justificación	Costo aprox.
Trípode robusto 1.6 m con cabezal flexible	Estabilidad de la C920 en captura y demos. Hoy se usa el de la laptop.	\$700 – \$1,200
Cable USB activo 5 m (con repetidor)	Permite colocar la cámara a 3–4 m del sujeto sin perder señal.	\$800 – \$5,000
SSD NVMe externo o interno 1 TB	Dataset + checkpoints + caché de embeddings para la sesión.	\$1,500 – \$2,500
Panel LED de iluminación regulable	Condiciones reproducibles para pruebas de robustez.	\$2,000 – \$16,500

5.2 Recomendado (mejora notable de calidad)

Ítem	Justificación	Costo aprox.
UPS / No-break 600–900 VA	Protege entrenamientos y corridas de GA largas (varias horas).	\$1,200 – \$2,000
Marca de piso (cinta + conos)	Define el corredor de captura, reduce varianza entre sesiones.	\$150 – \$300
Fondo neutro (tela 2×3 m, gris mate)	Mejora la calidad de la silueta (segmentación RVM).	\$400 – \$800
Cámara IP / segunda C920 de respaldo	Redundancia y posibilidad de capturar a 0° / 45° para fase evolutiva.	\$1,500 – \$2,500

5.3 Opcional (si el presupuesto lo permite)

Ítem	Justificación	Costo aprox.
GPU RTX 3060 12 GB o RTX 4060 8 GB	Acelera GA/CMA-ES y permite Tensor Cores (FP16) sin CUBLAS).	\$6,500 – \$9,500
Mini-PC / NUC para despliegue final	Demo en aula sin depender de la laptop personal.	\$8,000 – \$12,000
Monitor secundario 24"	Productividad durante depuración del pipeline en video.	\$2,500 – \$4,500

Resumen presupuestal estimado: indispensable ≈ \$3,300 – \$5,700 MXN; con recomendado, ≈ \$6,500 – \$11,300 MXN; con opcional completo, ≈ \$23,500 – \$37,300 MXN.

6. Limitaciones conocidas y riesgos

- **Dataset pequeño y monoangular:** 19 sujetos, todos a 90° lateral, en 2 sesiones. Generalización limitada a otras vistas o cámaras.
- **Par crítico *ricardomora* ↔ *hectorsanchez*:** mismo build corporal, falla en NN y NR. Documentado como caso aceptado.
- **Dependencia de calibración por cámara:** τ debe re-ajustarse si se cambia la cámara o el escenario (mitigable con GA reentrenando τ).
- **Sin Tensor Cores en la GTX 1650:** nos obliga a FP32; un upgrade a RTX 30/40 daría ~2× en throughput inmediato.
- **Dataset no es re-grabable a corto plazo:** evitar transformaciones destructivas; cualquier limpieza debe ser sobre copias.

7. Próximos pasos inmediatos (próximas 2 semanas)

Semana	Entregable	Responsable principal
1	Validación en vivo con $\tau=0.85$ sobre los 19 enrolados + 4 desconocidos	Equipo completo
1	Migración del log a SQLite con esquema definido	Aguilar / Cazarez
2	Implementación del wrapper evaluate(individual) → fitness	Espinoza
2	Piloto GA (DEAP) + piloto CMA-ES en RTX 2060	Vázquez / Aguilar
2	Manual de operación y guía de calibración	Cazarez / Espinoza

Criterio de cierre del proyecto: pipeline en vivo a ≥ 15 FPS, τ optimizado por GA/CMA-ES, log persistido en SQLite, manual entregado y demo funcional ante el grupo.

Apéndice A. Configuración activa del pipeline

Extracto del archivo configs/pipeline.yaml:

Sección	Parámetro	Valor
camera	target_fps	60 (pipeline limita a ~20)
detector	imgsz	640
embedder	compile	false (Triton no disponible en Windows)
matcher	tau	0.85
sequence	window	60 frames (~4 s @ 15 FPS)
sequence	stride	30 frames (50 % de solapamiento)

Checkpoint del modelo en producción: checkpoints/finetune/gaitbase_ft_multisession_best_iter1200.pt

Reportes técnicos previos: ver carpeta reports/ con la secuencia 01–07 (auditoría → fases), PHASE4_CIERRE_2026-04-28.md, y CHECKPOINT_2026-05-12.md (último checkpoint detallado).